

Hibernate a fondo: decisiones de mapeo y ciclo de vida

6. Hibernate a fondo: decisiones de mapeo y ciclo de vida

Hibernate a fondo: decisiones de mapeo y ciclo de vida

Ya sabes qué es un ORM y llevas usando sus anotaciones desde la primera actividad. Toca profundizar en lo que hasta ahora dabas por sentado: que esas anotaciones se podrían haber declarado de forma distinta, qué estados atraviesa un objeto mientras el ORM lo gestiona, y por qué tomaste ciertas decisiones de mapeo (IDENTITY, LAZY, cascade...) que hasta ahora no habías cuestionado.

Clase persistente y formas de declarar el mapeo

Una clase persistente (o entidad) es una clase Java cuyos objetos el ORM sabe guardar y recuperar de la base de datos. Existen dos formas históricas de declarar el mapeo entre una clase y una tabla:

Forma de mapeo	Cómo se declara	Estado hoy
Fichero XML	Documento aparte (<hibernate-mapping>, un <class> por entidad, un <property> por campo)	Legado — apenas se usa en proyectos nuevos
Anotaciones	Directamente sobre la propia clase (@Entity, @Column...)	El que usas tú desde la Actividad 1.1, y el estándar actual

Por qué ganaron las anotaciones

Las anotaciones ganaron terreno porque viven pegadas al propio código: cambias un campo y su mapeo a la vez, en el mismo fichero, sin mantener sincronizados un .java y un .xml por separado.

Los estados de un objeto en el ORM

Un objeto gestionado por un ORM pasa por distintos estados a lo largo de su vida:

- Transient: objeto Java recién creado, sin relación todavía con la base de datos — Hibernate no sabe que existe.
- Managed (o persistent): Hibernate lo está gestionando activamente — si cambias alguno de sus campos, detectará el cambio y lo guardará al finalizar.
- Detached: Hibernate ya no está gestionando el objeto — conserva sus datos, pero los cambios ya no se sincronizan automáticamente.
- Removed: marcado para eliminarse de la base de datos.

Crear una entidad nueva

```
Videojuego videojuego = new Videojuego(); // TRANSIENT
videojuego.setTitulo("Celeste");

Videojuego guardado = videojuegoRepository.save(videojuego);
```

Al utilizar `new`, el objeto solo existe en la memoria de Java: está en estado transient. Cuando se guarda, Hibernate empieza a gestionarlo y lo inserta en la base de datos.

Recoge siempre el objeto que devuelve save()

```
Videojuego guardado = videojuegoRepository.save(videojuego);
```

Ese objeto contiene el estado definitivo de la entidad, incluido el identificador generado por la base de datos.

Recuperar y modificar una entidad

```
@Transactional
public void cambiarPrecio(Long id, BigDecimal nuevoPrecio) {
    Videojuego videojuego = videojuegoRepository.findById(id)
        .orElseThrow();

    videojuego.setPrecio(nuevoPrecio);
}
```

La entidad obtenida mediante `findById()` está en estado managed mientras dura la transacción. Hibernate detecta el cambio de precio y lo guarda cuando la transacción termina, aunque no aparezca una nueva llamada a `save()`.

Cuando la transacción termina

Cuando la transacción finaliza y Hibernate deja de gestionar ese objeto, pasa a estado detached. Conserva todos sus datos, pero los cambios posteriores ya no se sincronizan automáticamente con la base de datos.

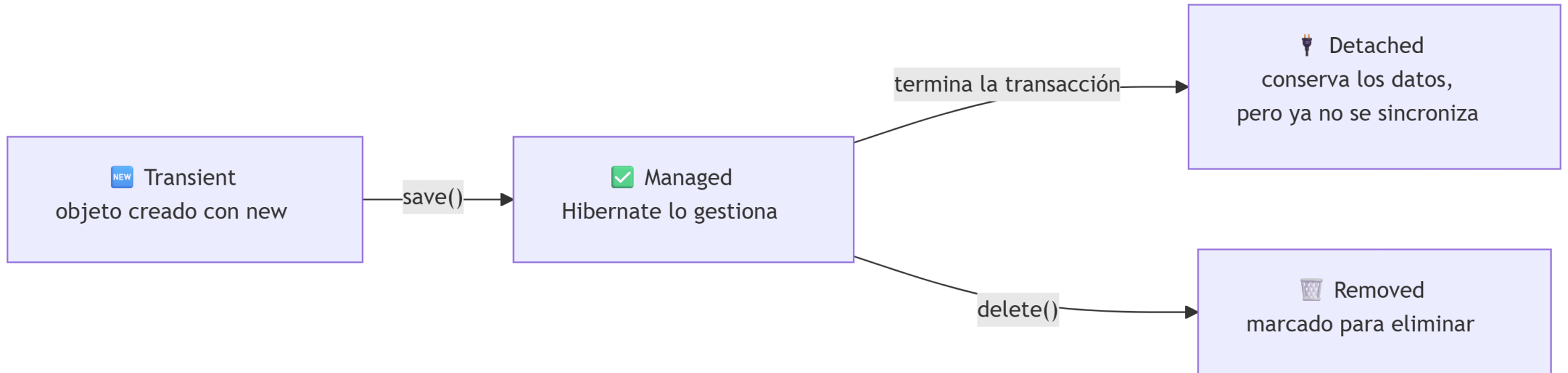
Eliminar una entidad

```
@Transactional
public void eliminar(Long id) {
    Videojuego videojuego = videojuegoRepository.findById(id)
        .orElseThrow();

    videojuegoRepository.delete(videojuego);
}
```

Al llamar a `delete()`, la entidad queda en estado `removed`: Hibernate la marca para eliminarla de la base de datos al confirmar la transacción.

Los cuatro estados, de un vistazo



`persist()` y `merge()`, ocultos tras `save()`

En JPA existen operaciones internas distintas para guardar entidades nuevas y entidades que ya existían: `persist()` y `merge()`. Spring Data JPA oculta esa diferencia detrás de `save()`. No necesitas decidir cuál utilizar en cada caso.

Para trabajar de forma segura, quédate con dos reglas:

- Si cargas una entidad dentro de una transacción, puedes modificarla y Hibernate detectará los cambios.
- Cuando llames a `save()`, utiliza el objeto que devuelve el repository.

¿Cuándo deja de estar managed?

Hibernate mantiene una zona de trabajo en la que controla los objetos que ha cargado o guardado. Esta zona se llama contexto de persistencia. Mientras un objeto está dentro de ese contexto, permanece managed: Hibernate detecta sus cambios y puede guardarlos automáticamente.

```
managed → Hibernate vigila los cambios  
detached → Hibernate ya no vigila los cambios
```

En los métodos anotados con `@Transactional`, lo habitual es que el objeto permanezca gestionado durante toda la transacción.

"Instalar" Hibernate no es un paso manual

A diferencia de lo que sugiere la palabra "instalar", en un proyecto Spring Boot no descargas ni configuras Hibernate por separado: viene incluido en la dependencia que ya conoces del Tema 1.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

spring-boot-starter-data-jpa trae Hibernate como implementación de JPA por defecto, además de Spring Data JPA — ya lo tienes en tu pom.xml desde la Actividad 1.1.

Decisiones de mapeo que ya has tomado, explicadas

Tu Videojuego y tu Estudio de la Actividad 1.1 ya llevan varias anotaciones concretas — las has escrito siguiendo el patrón de la teoría, sin que se justificara todavía cada elección. Toca cerrar ese hueco.

GenerationType.IDENTITY, ¿por qué esa estrategia?

GenerationType tiene cinco valores. Todos resuelven el mismo problema —generar un identificador único—, pero lo hacen de formas diferentes:

Estrategia	Cómo genera el id	Cuándo conviene
IDENTITY	Delega en una columna autoincremental de la propia base de datos.	La que usas en Videojuego y Estudio, con identificadores Long.
SEQUENCE	Utiliza una secuencia de la base de datos que Hibernate consulta antes de insertar.	Cuando necesitas reservar identificadores por adelantado.
TABLE	Simula una secuencia mediante una tabla auxiliar gestionada por Hibernate.	Cuando el gestor no dispone de columnas identity ni secuencias.
UUID	Genera un identificador UUID (550e8400-e29b-...).	Cuando interesa generar ids únicos sin depender de una secuencia central.
AUTO	Deja que Hibernate elija la estrategia más adecuada.	Cuando prefieres delegar la elección en el ORM.

En este proyecto, IDENTITY

En este proyecto continuarás utilizando IDENTITY, porque tus entidades tienen un identificador numérico Long y PostgreSQL puede generarlo automáticamente. UUID es otra posibilidad útil, pero requeriría declarar el identificador con el tipo UUID:

```
@Id
@GeneratedValue(strategy = GenerationType.UUID)
private UUID id;
```

TABLE, en concreto: la tabla auxiliar que crea Hibernate

TABLE es de las estrategias más difíciles de visualizar, porque no usa ninguna función nativa del motor — Hibernate crea una tabla normal, con una fila por entidad, y hace un SELECT + UPDATE dentro de una transacción cada vez que necesita un id nuevo:

```
CREATE TABLE hibernate_sequence (  
    entidad VARCHAR(255) PRIMARY KEY,  
    valor_actual BIGINT  
);  
  
SELECT valor_actual FROM hibernate_sequence WHERE entidad = 'videojuego'; -- 1. lee  
UPDATE hibernate_sequence SET valor_actual = valor_actual + 1 WHERE entidad = 'videojuego'; -- 2.  
incrementa
```

TABLE (cont.) — por qué suele ser la menos eficiente

Es literalmente lo mismo que hace SEQUENCE por dentro — la diferencia es que SEQUENCE es un objeto del motor, optimizado para que muchas inserciones a la vez no se bloqueen entre sí; TABLE es una tabla corriente, y ese SELECT/UPDATE se bloquea como cualquier otra fila. Con muchas inserciones simultáneas se convierte en un cuello de botella.

@Column(precision, scale), y por qué importa en dinero

Para guardar dinero tenías, en realidad, tres alternativas:

Alternativa	Cómo funciona	Por qué no se usó (o sí)
BigDecimal sin precision/scale	Hibernate elige un tipo de columna genérico	Puede admitir más decimales de los que tiene sentido para un precio
BigDecimal con precision/scale (la elegida)	Tú fijas exactamente cuántos dígitos y decimales admite la columna	Precisión explícita, sin depender del valor por defecto del motor
Entero en céntimos (Long)	El dinero se guarda como entero (250099 = 2500,99€)	Elimina el problema de raíz, a cambio de más código al convertir a euros

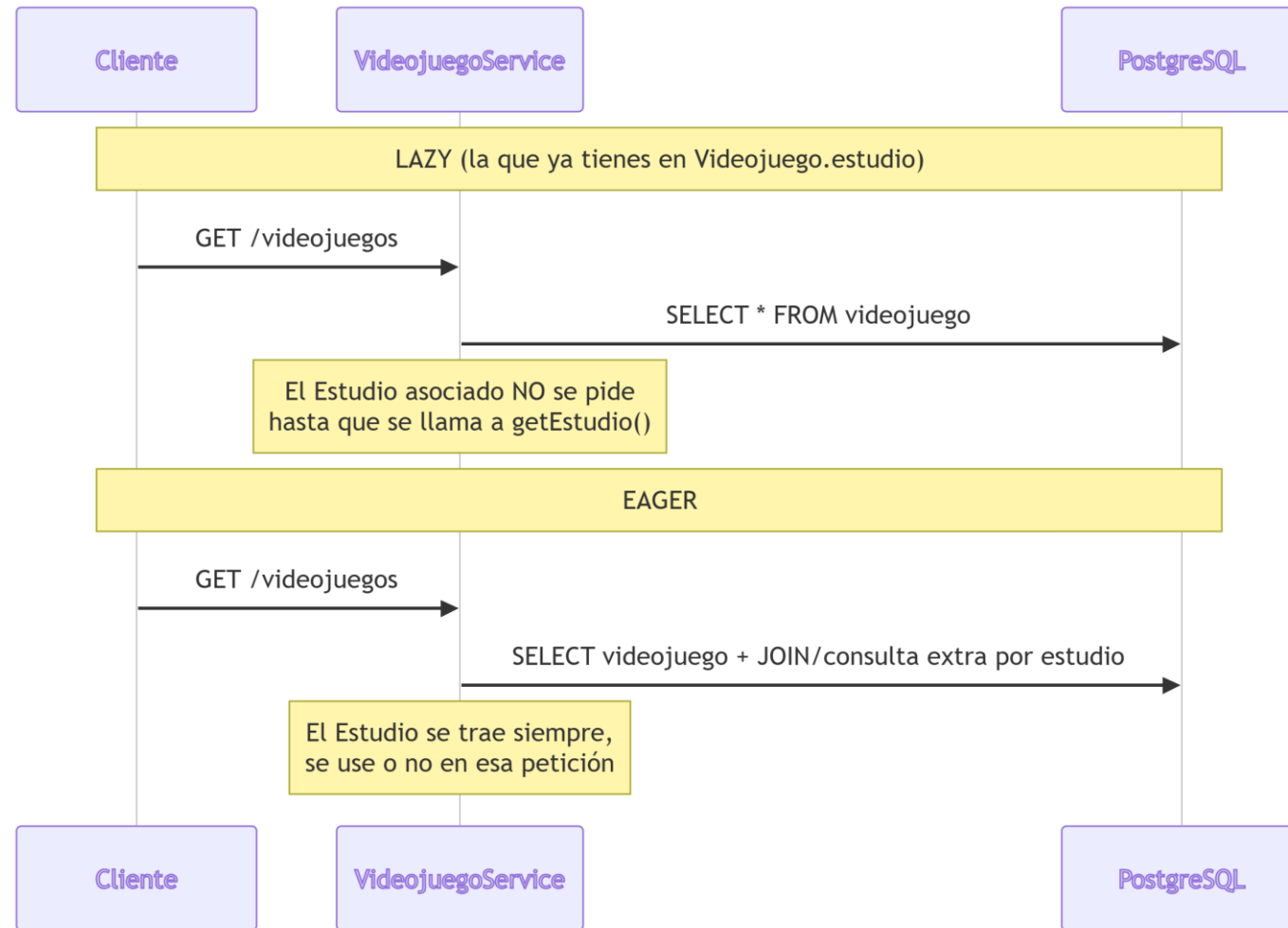
Puedes comprobar la diferencia tú mismo

Quita temporalmente `@Column(precision = 10, scale = 2)` de precio, borra la tabla y reinicia con `show-sql: true` activo — verás que el `create table` genera un tipo de columna distinto. En una columna que guarda dinero, esa precisión no es un detalle decorativo.

FetchType.LAZY frente a EAGER

Puedes comprobarlo activando el log de SQL (`logging.level.org.hibernate.SQL:DEBUG`) y contando cuántas consultas distintas aparecen al llamar a `GET /api/v1/videojuegos`.

LAZY frente a EAGER, en número real de consultas



Lo que cambia, en la práctica

Con LAZY, Hibernate no trae el Estudio asociado hasta que de verdad lo pides. Con EAGER, lo trae siempre en la misma consulta o en una adicional inmediata — más consultas (o joins más pesados) por cada petición, incluso cuando el cliente nunca llega a mirar ese dato.

cascade / orphanRemoval, y qué otras opciones había

CascadeType.ALL no es la única opción — es un atajo que combina otras cinco por separado:

Valor de cascade	Qué propaga de Estudio hacia sus Videojuego
(sin cascade)	Nada — cada operación sobre un Videojuego la harías tú mismo, uno a uno
PERSIST	Guardar el Estudio guarda también los videojuegos nuevos que le hayas añadido
MERGE	Actualizar el Estudio sincroniza también los cambios de sus videojuegos ya existentes
REMOVE	Borrar el Estudio borra también todos sus videojuegos
REFRESH / DETACH	Recargar o desvincular el Estudio hace lo mismo con sus videojuegos
ALL (la elegida)	Las cinco anteriores combinadas, en una sola palabra

Por qué se eligió ALL

Con tu configuración actual (`cascade = CascadeType.ALL`, `orphanRemoval = true` en `Estudio.videojuegos`), borrar un estudio borra en cascada sus videojuegos.

Podrías haber elegido, por ejemplo, `cascade = {PERSIST, MERGE}` sin `REMOVE`: guardar y actualizar se propagarían igual, pero borrar un Estudio con videojuegos asociados fallaría por la clave foránea — obligándote a decidir explícitamente qué hacer con ellos.

Se eligió `ALL` porque, en este proyecto, un videojuego sin su estudio no tiene sentido de negocio: no hay ningún caso en que interese conservarlo huérfano.

orphanRemoval, una decisión aparte

orphanRemoval es una decisión aparte, binaria — true (la elegida) o false (el valor por defecto si no lo escribes):

Si orphanRemoval es...	Qué pasa al quitar un Videojuego de la lista (sin borrar el Estudio)
true (la elegida)	Hibernate lo detecta como huérfano y lo elimina de la base de datos por su cuenta
false (por defecto)	El videojuego se queda en la base de datos, sin ningún Estudio que lo referencie

Dónde se declara la cascada, y por qué

`orphanRemoval = true` se utiliza cuando el objeto hijo no tiene sentido por separado: si deja de pertenecer al objeto padre, también debe desaparecer de la base de datos.

La cascada se declara en el lado Estudio → Videojuego (el lado "uno" de la relación) porque no tendría sentido al revés: borrar un solo videojuego no debería llevarse por delante el estudio entero.

Configuración avanzada: cuando el mapeo no es directo

Hibernate necesita, a veces, ayuda extra para mapear tipos sin correspondencia directa con una columna estándar — por ejemplo, para cifrar un campo o dar formato propio a un tipo que Hibernate no reconoce. Esa ayuda se declara con un `@Converter` (una clase `AttributeConverter`): no necesitas escribir uno todavía, basta con saber que es ahí donde se completa el mapeo automático.

ddl-auto: configuración del ORM, no del conector

Ya has usado `spring.jpa.hibernate.ddl-auto: update` desde la Actividad 1.1, pero desde el ángulo de "cómo se crea la tabla".

Con la distinción entre conector y ORM que ya conoces: la conexión (usuario, contraseña, URL) es configuración del conector; qué hace Hibernate con las entidades que declaras (crear tablas, validarlas, no tocar nada) es configuración del ORM — y vive en esa misma propiedad, `ddl-auto`.

Las propiedades de configuración del ORM que más usarás

Propiedad	Qué controla
spring.jpa.hibernate.ddl-auto	Si Hibernate crea, actualiza o solo valida las tablas al arrancar.
spring.jpa.show-sql	Si Hibernate imprime en consola el SQL real que ejecuta por debajo.
spring.jpa.properties.hibernate.format_sql	Si ese SQL impreso se formatea legible, en varias líneas.

El otro lado de lo automático: data.sql

ddl-auto se ocupa de la estructura de la base de datos: crea o actualiza las tablas a partir de las entidades. Para introducir datos iniciales puedes crear un fichero src/main/resources/data.sql, con sentencias INSERT normales:

```
INSERT INTO estudio (nombre) VALUES ('Nintendo');  
INSERT INTO estudio (nombre) VALUES ('Supergiant Games');
```

Como el proyecto usa PostgreSQL, hacen falta dos propiedades más

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
      defer-datasource-initialization: true

  sql:
    init:
      mode: always
```

- `spring.sql.init.mode: always` hace que Spring Boot ejecute `data.sql` también con PostgreSQL.
- `spring.jpa.defer-datasource-initialization: true` espera a que Hibernate haya preparado las tablas antes de ejecutar los INSERT.

El orden de arranque

Hibernate prepara las tablas → Spring Boot ejecuta data.sql → arranca la aplicación

⚠ data.sql se ejecuta en cada arranque

Los datos del fichero deben estar preparados para no provocar errores por insertar varias veces los mismos registros. En la Actividad 1.5 utilizarás este mecanismo para disponer rápidamente de suficientes datos con los que probar la paginación.

Ideas clave

- El mapeo se declara con anotaciones sobre la propia entidad (el XML de mapeo es el enfoque legado).
- Los estados de un objeto en el ORM: transient (sin relación con la BD) → managed (gestionado, se sincroniza solo) → detached (ya no se sincroniza) / removed (marcado para borrar).
- "Instalar" Hibernate en Spring Boot es, en la práctica, añadir spring-boot-starter-data-jpa.
- IDENTITY delega la generación del id en la propia base de datos; SEQUENCE permite reservar ids por adelantado. precision/scale fijan el tipo exacto de columna.

Ideas clave (cont.)

- LAZY difiere la carga de una relación hasta que se pide de verdad; EAGER la trae siempre. `cascade/orphanRemoval` propagan borrados y limpian huérfanos automáticamente.
- `ddl-auto` es configuración del ORM (qué hace Hibernate con tus entidades), distinta de la configuración del conector (cómo se conecta); `show-sql/format_sql` muestran el SQL real generado.
- Para ejecutar un `data.sql` sobre PostgreSQL se utilizan `spring.sql.init.mode: always` y `spring.jpa.defer-datasource-initialization: true`.